COMP/EECE 7/8740 Neural Networks

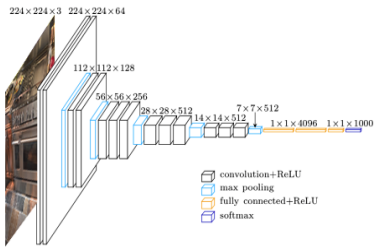
Topics:

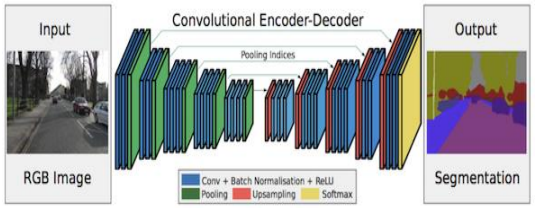
- Convolutional Neural Networks object detection and localization
  - Detection models
    - Faster R-CNN
    - SSD
    - YOLO
- Anchor free models
- Practical demos (YOLO).

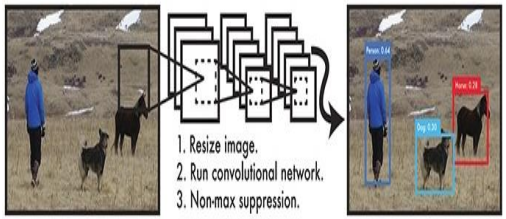
Md Zahangir Alom
Department of Computer Science
University of Memphis, TN

Deep CNN models and applications

- Classification
- Segmentation
- Detection







Models for classification:

- AlexNet
- VGG Net
- GoogleNet
- ResNet
- DenseNet
- FractalNet
- CapsuleNet
- IRRCNN

Models for Segmentation:

- FCN
- SegNet
- Dialted Convolution
- RefineNet
- Pyramid Scene Parsing (PSP):PSPNet
- DeepLab
- U-Net

Models for Detection:

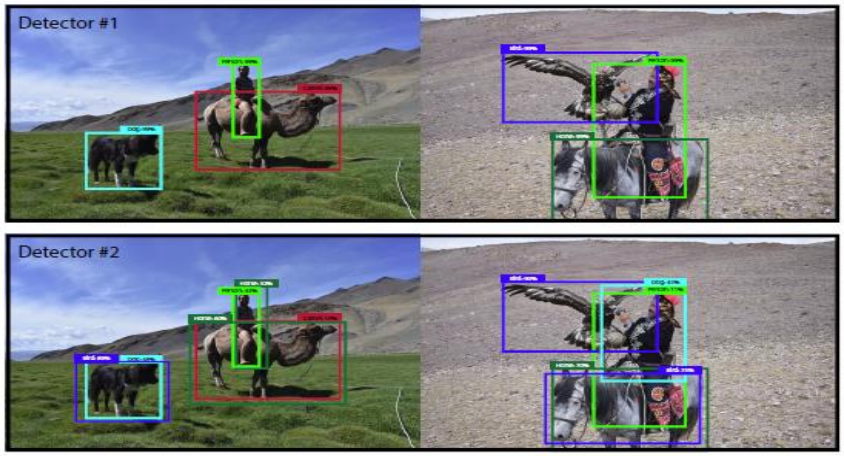
- Region based CNN (RCNN)
- Fast RCNN
- Faster RCNN
- Mask RCNN
- You Only Look Once (YOLO)
- Single Short Multibox Detector (SSD)

Seg.: http://blog.qure.ai/notes/semantic-segmentation-deep-learning-review

Detection: https://towardsdatascience.com/

Object detection with Deep Learning

- Object detection involves **not only recognizing** and **classifying every object in an image**, but ***localizing*** **each one by drawing the appropriate bounding box** on it.
- This makes **object detection a significantly harder** task image classification.
- **Challenges:** locations, scales and aspect ratios for the search of the objects



How end-to-end object detector works?

The way algorithm works is the following:

Step 1: Make a window of size much smaller than actual image size. Crop it and pass it to ConvNet (CNN) and make the predictions.

Step 2: Keep on sliding the window and pass the cropped images into ConvNet.

Step 3: Repeat steps 1 and 2 again for a bit bigger window size and make the predictions.

Step 4: Finally, you will have a set of cropped regions which will have some object, together with class and bounding box of the object.

Drawbacks:

- Computationally very expensive
- Difficult to define proper bounding box (objects may rectangular or squire)

Solutions:

- Actual implementation does use cropped images (1x1 conv. Kernel same as window size)

R-CNN → Faster R-CNN

- Faster R-CNN was **a canonical model** for deep learning-based object detection.
- Unfortunately, we can't really begin to understand Faster R-CNN **without understanding its own predecessors, R-CNN and Fast R-CNN**

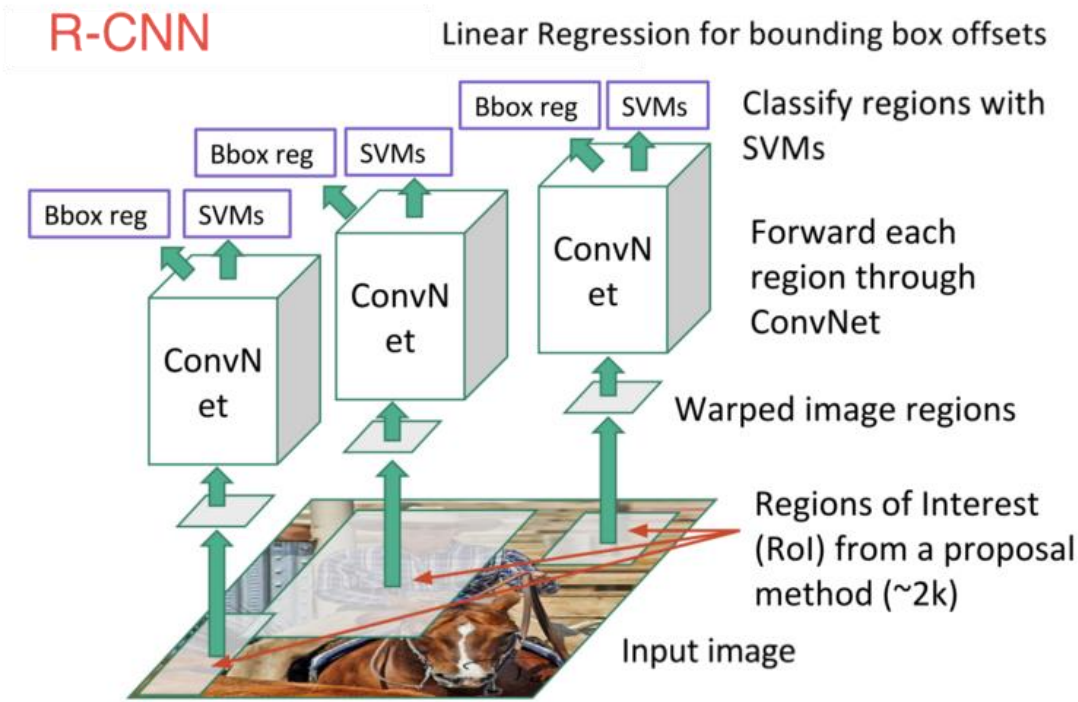
- R-CNN is the grand-daddy of Faster R-CNN. In other words, R-CNN *really kicked* things off.

Region-based Convolutional Neural Network (R-CNN)

- R-CNN, consisted of 3 simple steps:
  - Scan the input image for possible objects using an algorithm called **Selective Search**, generating ~2K **region proposals**
  - Apply a convolutional neural net (CNN) on top of each of these region proposals
  - Take the output of each **CNN** and feed it into
    - an SVM to classify the region and
    - a linear regressor to tighten the bounding box of the object, if such an object exists.

RCNN

- Three steps are illustrated in the image below:



- Propose regions, then extract features, and then classify those regions based on their features.
- **Problem has turned object detection into an image classification**.

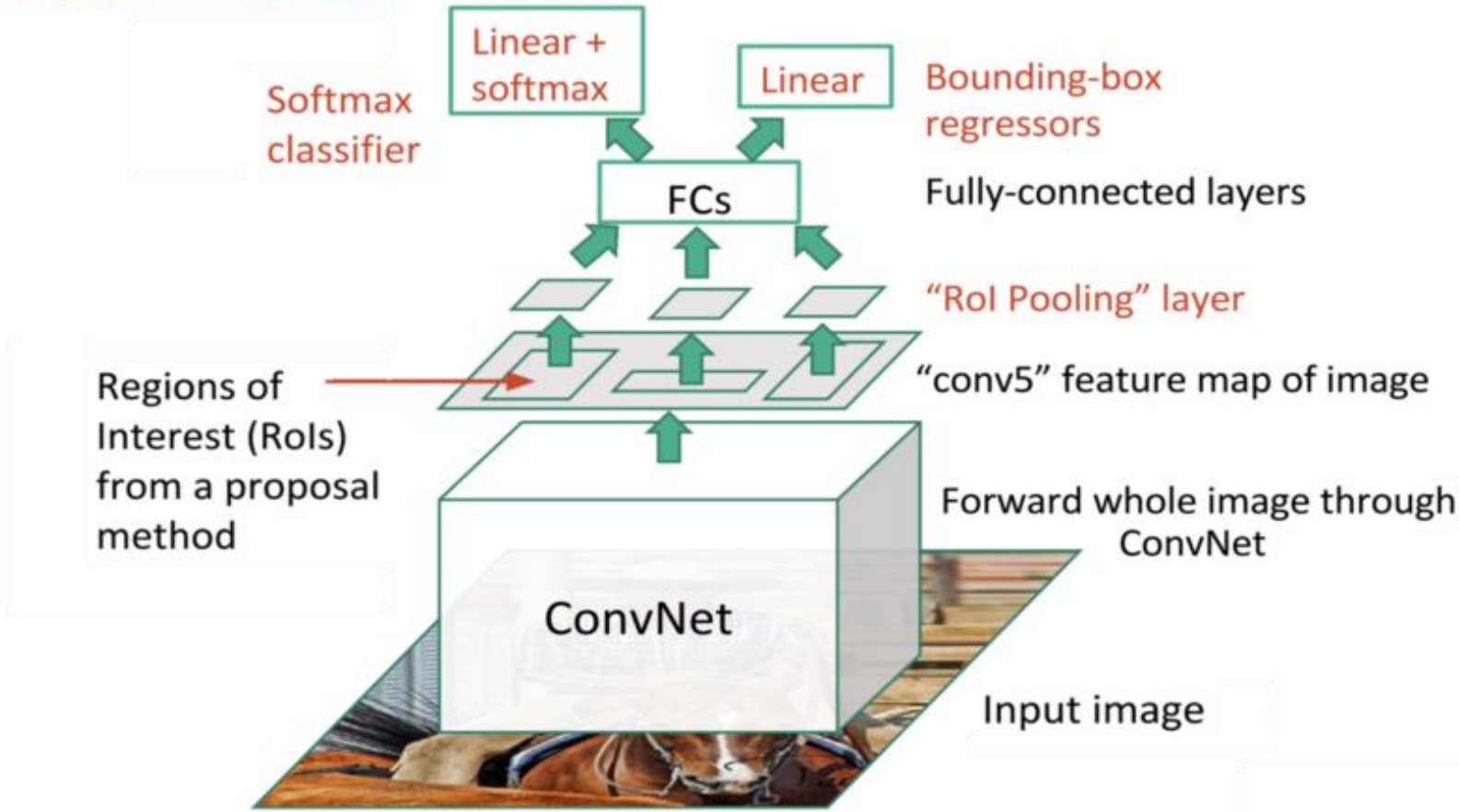
Fast R-CNN

- **R-CNN's immediate descendant was Fast-R-CNN**. Fast R-CNN resembled the original in many ways, but improved on its detection speed through two main augmentations:
  - Performing feature extraction over the image before proposing regions, thus **only running one CNN over the entire image instead of 2K** CNN's over 2K overlapping regions
  - **Replacing the SVM with a softmax layer**, thus extending the neural network for predictions instead of creating a new model

Girshick, Ross. "Fast r-cnn." In *Proceedings of the IEEE international conference on computer vision*, pp. 1440-1448. 2015.

https://towardsdatascience.com/deep-learning-for-object-detection-a-comprehensive-review-73930816d8d9

Fast R-CNN



- Fast R-CNN **performed much better in terms of speed compared to RCNN**

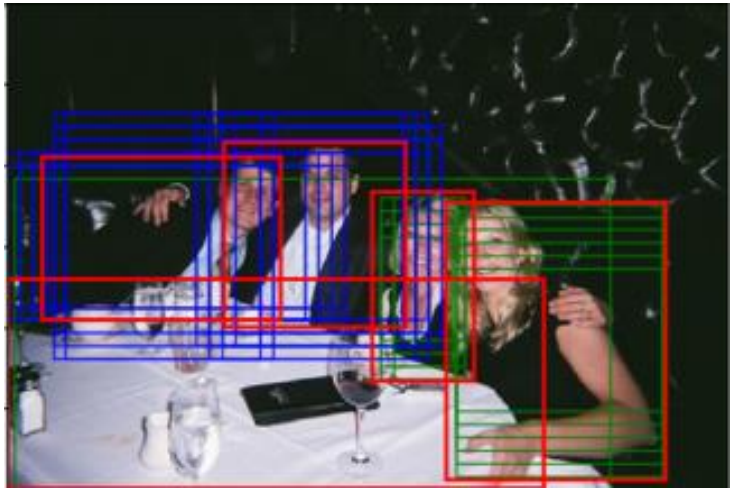
- **One big bottleneck remaining:** the selective search algorithm for generating region proposals.

Faster R-CNN = RPN + Fast R-CNN

- Faster R-CNN: the main insight of Faster R-CNN was to replace the **slow selective search algorithm with a fast neural net.**
- Specifically, it introduced the **region proposal network (RPN).**

Faster R-CNN: **PRN**

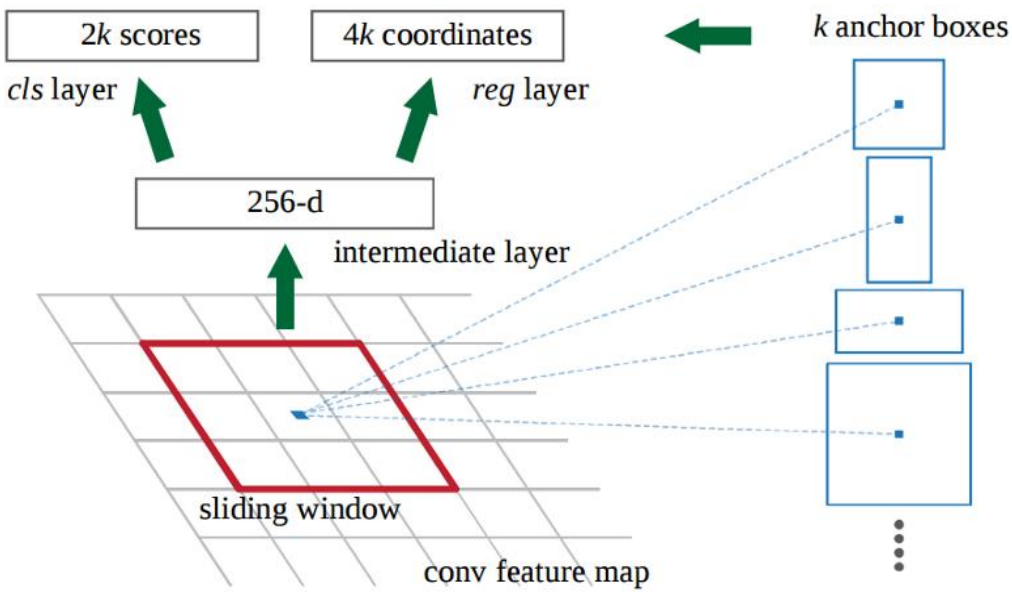
- Here's how the **RPN worked**:
  - At the last layer of an initial CNN, **a 3x3 sliding window moves across the feature map** and maps it to a lower dimension (e.g. 256-d)
  - For each sliding-window location, it **generates *multiple* possible regions based on *k* fixed-ratio anchor boxes**



Anchor boxes

Faster R-CNN:**RPN**

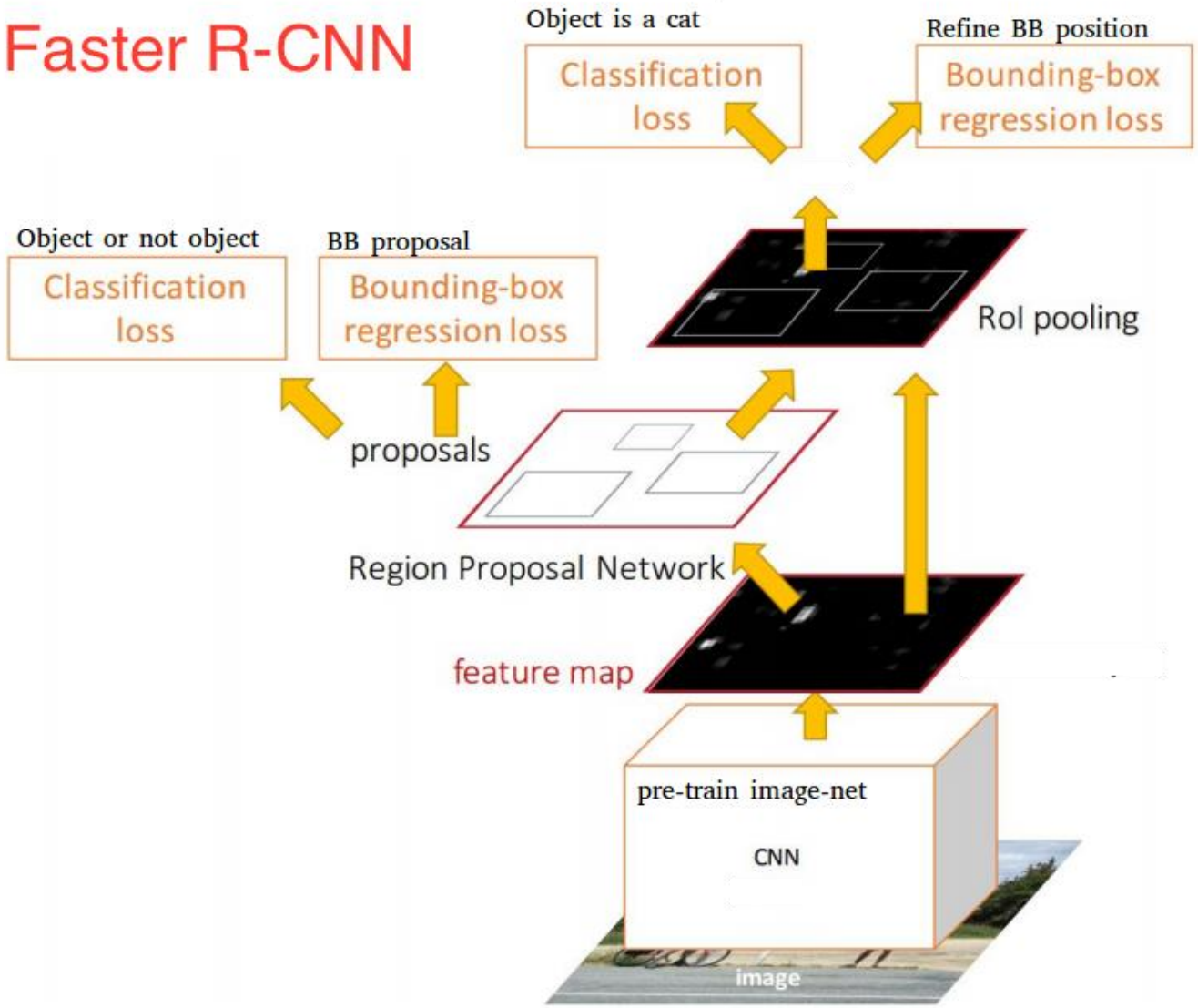
- From each location of last feature map and consider *k* different boxes centered around it: **a tall box, a wide box, a large box, etc.**
- For each of those boxes:
  - **whether or not the window contains** an object, and
  - what the **coordinates for that box are?**



Faster R-CNN = RPN + Fast R-CNN

- **RPN** :
  - The 2*k* scores represent the softmax probability of each of the ***k* bounding boxes being on "object."**
  - RPN outputs bounding box coordinates**: its sole job is still proposing object regions (not classify)**
  - **If an anchor box has an "objectness" score above a certain threshold**, that box's coordinates get passed forward as a region proposal.
- Once we have region proposals, we feed them straight into a Fast R-CNN.
  - A pooling layer, some fully-connected layers, and finally a softmax classification layer and bounding box regressor are added

Faster R-CNN = RPN + Fast R-CNN



Faster R-CNN

- Faster R-CNN achieved much **better speeds and a state-of-the-art accuracy.**
- Although future models did a lot to increase detection speeds, **few models managed to outperform Faster R-CNN by a significant margin.**
- Faster R-CNN may not be the simplest or fastest method for object detection, **but it is still one of the best performing.**

Region-based Fully Convolutional Net (R-FCN)

- R-FCN, or **R**egion-based **F**ully **C**onvolutional **N**et, shares 100% of the computations across every single output.

- **Objective to learn:**
  - **Location invariance** in a model: regardless of where the cat appears in the image, we want to classify it as a cat.
  - **Location variance:** if the cat is in the top left-hand corner, we want to draw a box in the top left-hand corner.

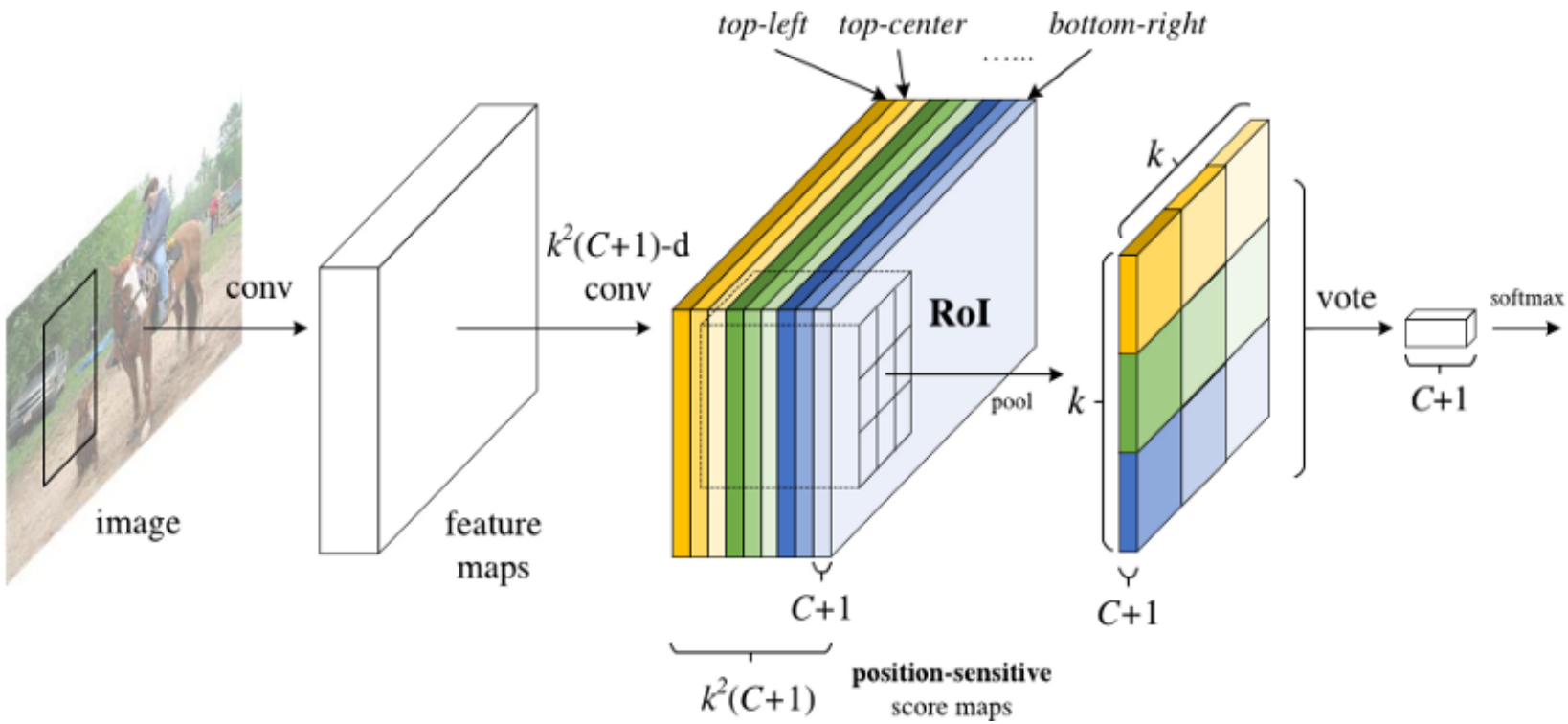
- R-FCN's solution: position-sensitive score maps.

R-FCN

- R-FCN works as follows:
  - Run a CNN (in this case, ResNet) over the input image
  - Add a fully convolutional layer to generate a **score bank** of the aforementioned **"position-sensitive score maps."** $k^2(C+1)$ score maps, with k^2 representing the number of relative positions to divide an object (e.g. 3^2 for a 3 by 3 grid) and **C+1 representing the number of classes plus the background**.
  - Run a fully convolutional **region proposal network (RPN)** to **generate regions of interest (RoI's)**
    - For each RoI, divide it into the same k^2 "bins" or subregions as the score maps
    - For each bin, check the **score bank to see if that bin matches the corresponding position of some object.** For example, if I'm on the "upper-left" bin, This process is repeated for each class.
    - Once each of the k^2 bins has **an "object match" value for each class**, average the bins to get a single score per class.
  - **Classify the RoI with a softmax** over the remaining C+1 dimensional vector

https://towardsdatascience.com/deep-learning-for-object-detection-a-comprehensive-review-73930816d8d9

R-FCN



- R-FCN is much easier to understand when you can visualize what it's doing. Example of an R-FCN in practice, detecting an object.

R-FCN in practice, detecting a baby

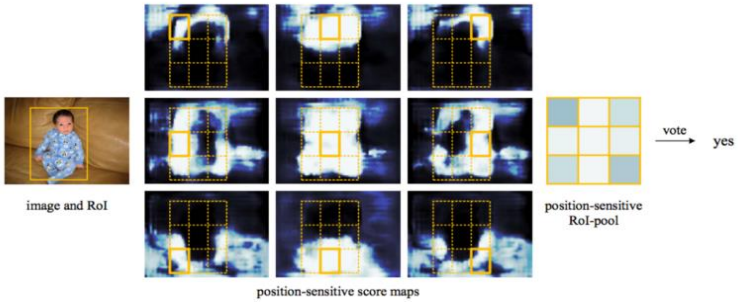


Figure 3: Visualization of R-FCN ($k \times k = 3 \times 3$) for the *person* category.

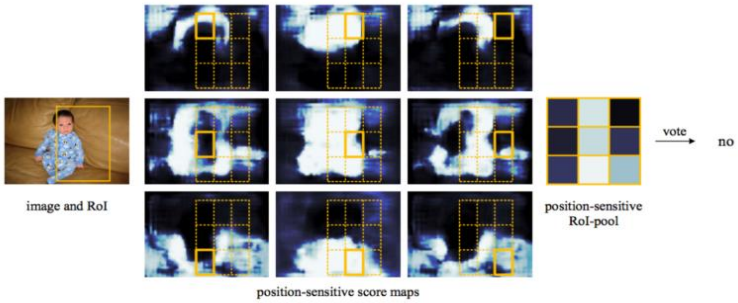


Figure 4: Visualization when an RoI does not correctly overlap the object.

- R-FCN considers each region proposal, divides it up into sub-regions, and iterates over the sub-regions asking: **"does this look like the top-left of a baby?", "does this look like the top-center of a baby?" "does this look like the top-right of a baby?",** etc.
- **Repeats this for all possible classes**.
- **If enough of the sub-regions say "yes**, I match up with that part of a baby!",
- The **RoI gets classified as a baby after a softmax over all the classes**.

Single-Shot Detector (SSD)

- **S**ingle-**S**hot **D**etector. Like R-FCN
- Provides **enormous speed gains over Faster R-CNN**, but does so in a markedly different manner.
- Previous two models performed
  - **Region proposals** and **region classifications**.

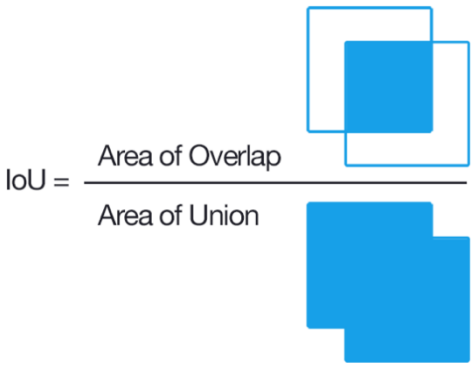
- **SSD** does the two in a "single shot," simultaneously predicting the bounding box and the class as it processes the image.

SSD-Algorithm

- Given an input image and a set of ground truth labels, SSD does the following:
  - Pass the image through a series of convolutional layers, yielding several sets of feature maps at different scales (e.g. **10x10, then 6x6, then 3x3**, etc.)
  - For each location in *each* of these feature maps, use a 3x3 convolutional filter to evaluate a small set of default bounding boxes **equivalent to Faster R-CNN's anchor boxes**.
  - For each box, simultaneously predict (4+*p* dimensional vector).
    a) The bounding box offset (4 box offset coordinates)
    b) The *p* class probabilities (Softmax)
  - During training, **match the ground truth box with these predicted boxes based on IoU.**
  - The best predicted box will be labeled a "positive," along with all other boxes that have an IoU with the truth >0.5.

IOU : performance metrics

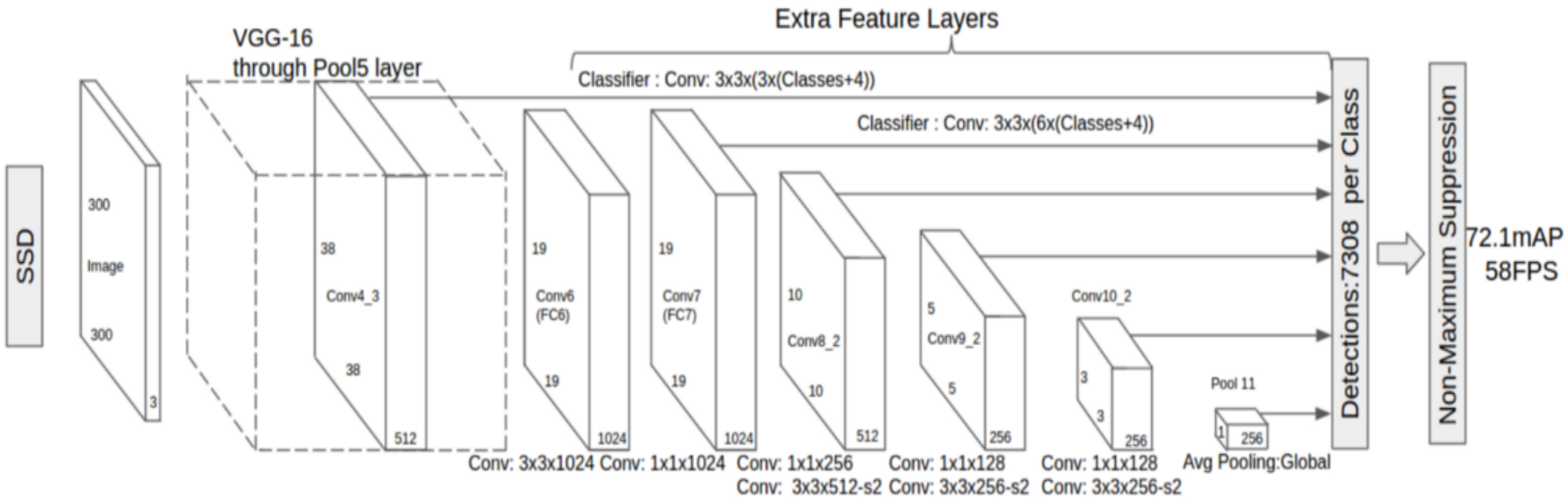
- Intersection over Union (IoU):



An example of computing Intersection over Unions for various bounding boxes.

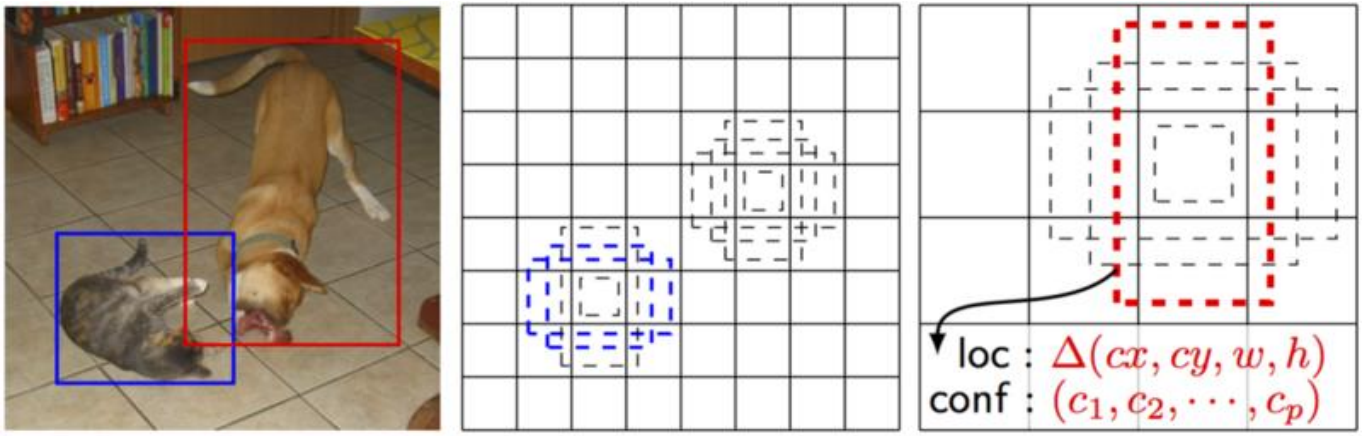
SSD-Architecture

- The **"extra feature layers"** at the end that scale down in size.
- Varying-size feature maps help **capture objects of different sizes**.



SSD Challenges

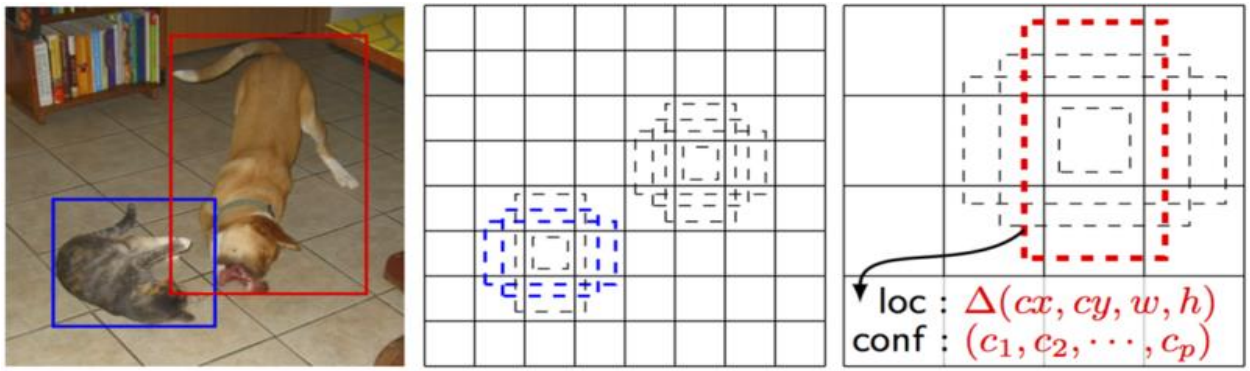
- SSD sounds straightforward, but **training it has a unique challenge.**
  - Classify and draw bounding boxes from every single position in the image, using **multiple different shapes, at several different scales.**
  - Generate **a much greater number of bounding boxes than the other models**, and nearly all of the them are negative examples.



(a) Image with GT boxes (b) 8×8 feature map (c) 4×4 feature map

SSD

- To fix this imbalance, SSD does two things:
  - Firstly, it uses **<u>non-maximum suppression</u>** to group together highly-overlapping boxes into a single box.
    - if four boxes of similar shapes, sizes, etc. contain the same dog, NMS would keep the one with the highest confidence and discard the rest.
  - Secondly, the model uses a technique called **<u>hard negative mining</u>** to balance classes during training.
    - only a subset of the negative examples **with the highest training loss (i.e. false positives)** are used at each iteration of training.
    - SSD keeps a 3:1 ratio of negatives to positives.



(a) Image with GT boxes (b) 8×8 feature map (c) 4×4 feature map

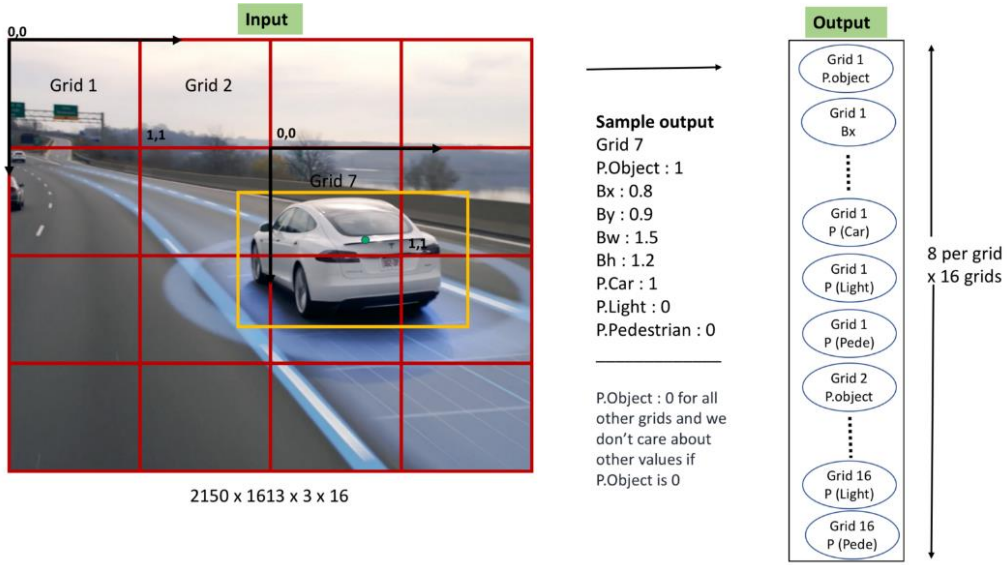
SSD-summary

- SSD is not so different from the first two models.
- **Skips the "region proposal" step**, instead considering every single bounding box in every location of the image simultaneously with its classification.
- SSD does **everything in one shot**, it is the <u>fastest of the three models</u>, and
- Still performs quite comparably.

You Only Look Once (YOLO)

Step 1: Divide the image into multiple grids.

Step 2. Label the training data as shown in the figure. For C is number of unique objects, S*S is number of grids of input image output vector : S*S*(C+5). For 4x4 grid, target vector is 4*4*(3+5)=128 and are training for 3 unique classes: light, pedestrian, and car.



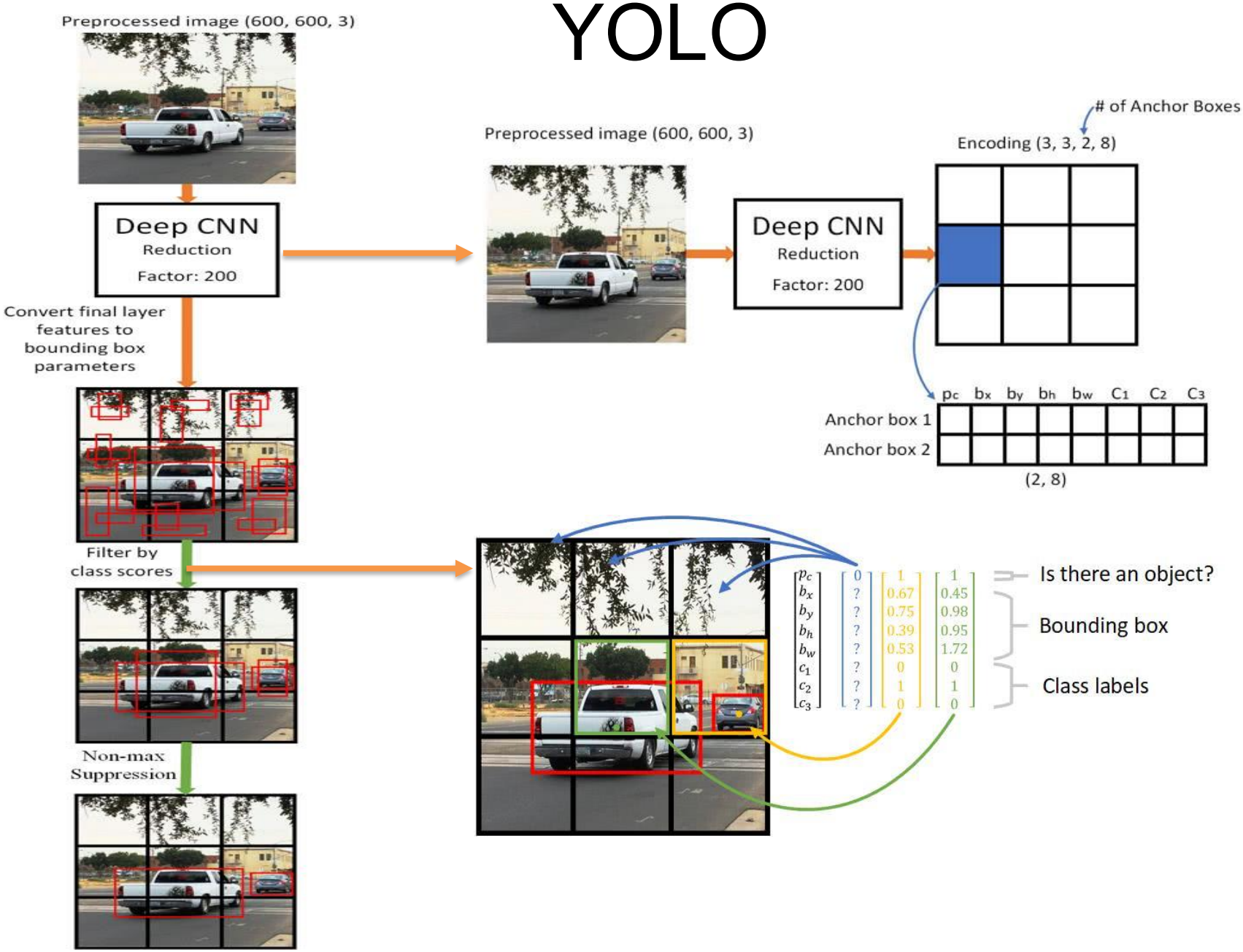
Step 3. Make DCNN model with loss function as error between output activations and label vector and **model predicts the output of all the grids in just one forward pass.**

Step 4. Object being present in a grid cell is determined by (P.Object) that not allow one object to be counted multiple times in different grids.

Step 5. Anchor boxes: (c+5)*A labels for each grid (same grid have **multiple anchor boxes**) and **non-max suppression.**

https://towardsdatascience.com/evolution-of-object-detection-and-localization-algorithms-e241021d8bad

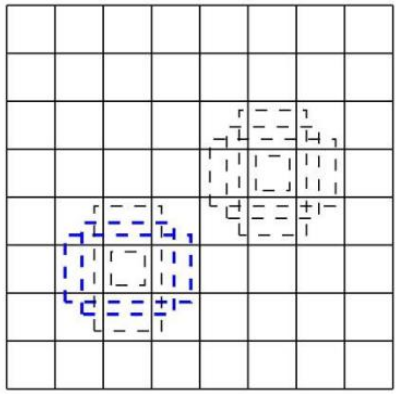
YOLO



https://towardsdatascience.com/yolo-you-only-look-once-real-time-object-detection-explained-492dc9230006zaaraa

Recent trends: Anchor free detector

- FCOS(FCOS: Fully Convolutional One-Stage Object Detection), which outperform anchor-based model
- FoveaBox : an accurate, flexible and completely anchor-free framework for object detection
- CenterNet
- ExtremeNet
- CornerNet-Lite
- DenseBox



anchor boxes

Mostly attention-based model !

Zhu, Chenchen, Yihui He, and Marios Savvides. "Feature selective anchor-free module for single-shot object detection." *arXiv preprint arXiv:1903.00621* (2019).

Kong, Tao, Fuchun Sun, Huaping Liu, Yuning Jiang, and Jianbo Shi. "FoveaBox: Beyond Anchor-based Object Detector." *arXiv preprint arXiv:1904.03797* (2019).

Zhou, Xingyi, Jiacheng Zhuo, and Philipp Krahenbuhl. "Bottom-up object detection by grouping **extreme** and center points." In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 850-859. 2019.

Law, Hei, Yun Teng, Olga Russakovsky, and Jia Deng. "CornerNet-Lite: Efficient Keypoint Based Object Detection." *arXiv preprint arXiv:1904.08900* (2019).

Why Anchor free detectors?

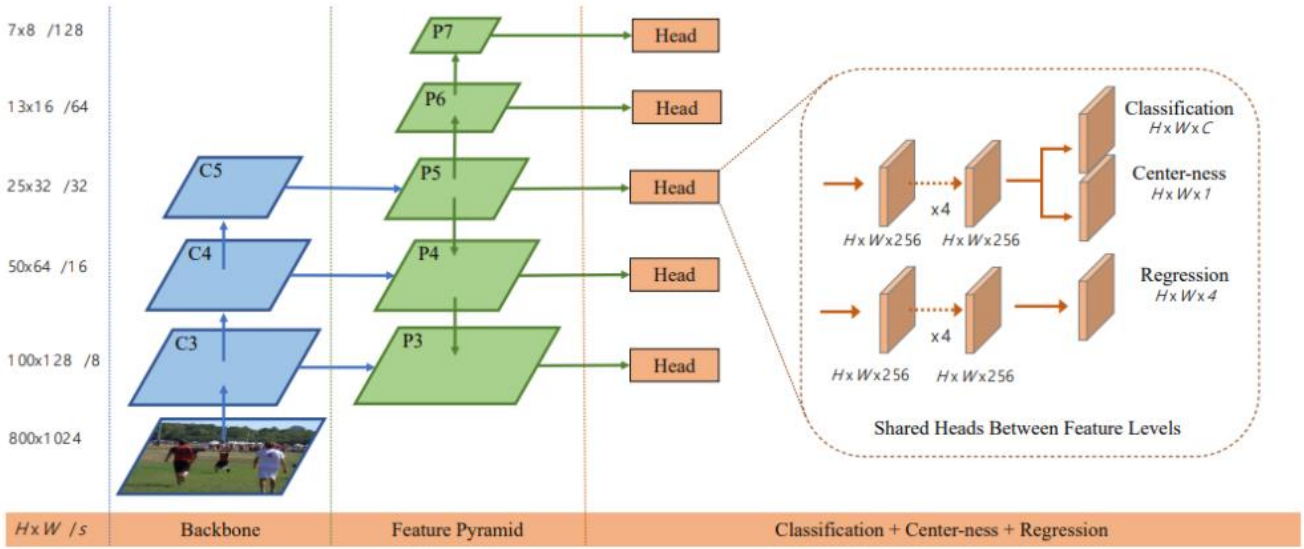
- Anchor based object detection has some unsolved issues:
  - **The numbers of hyper parameters to set**

    Anchor based needed to set anchor for manually. Every person tried to tune hyper parameters knows how suffer it is to decide aspect ratio and for each feature maps**. Anchor free don't need that.**

  - **Imbalances between positive and negative samples**

    Anchor based models set positive box (box with object) by calculating IOU between anchor box and ground truth box. **Example:** SSD produced roughly 9,000 boxes at time and labeled only some of them to positive boxes while labeled most boxes as negative(background).

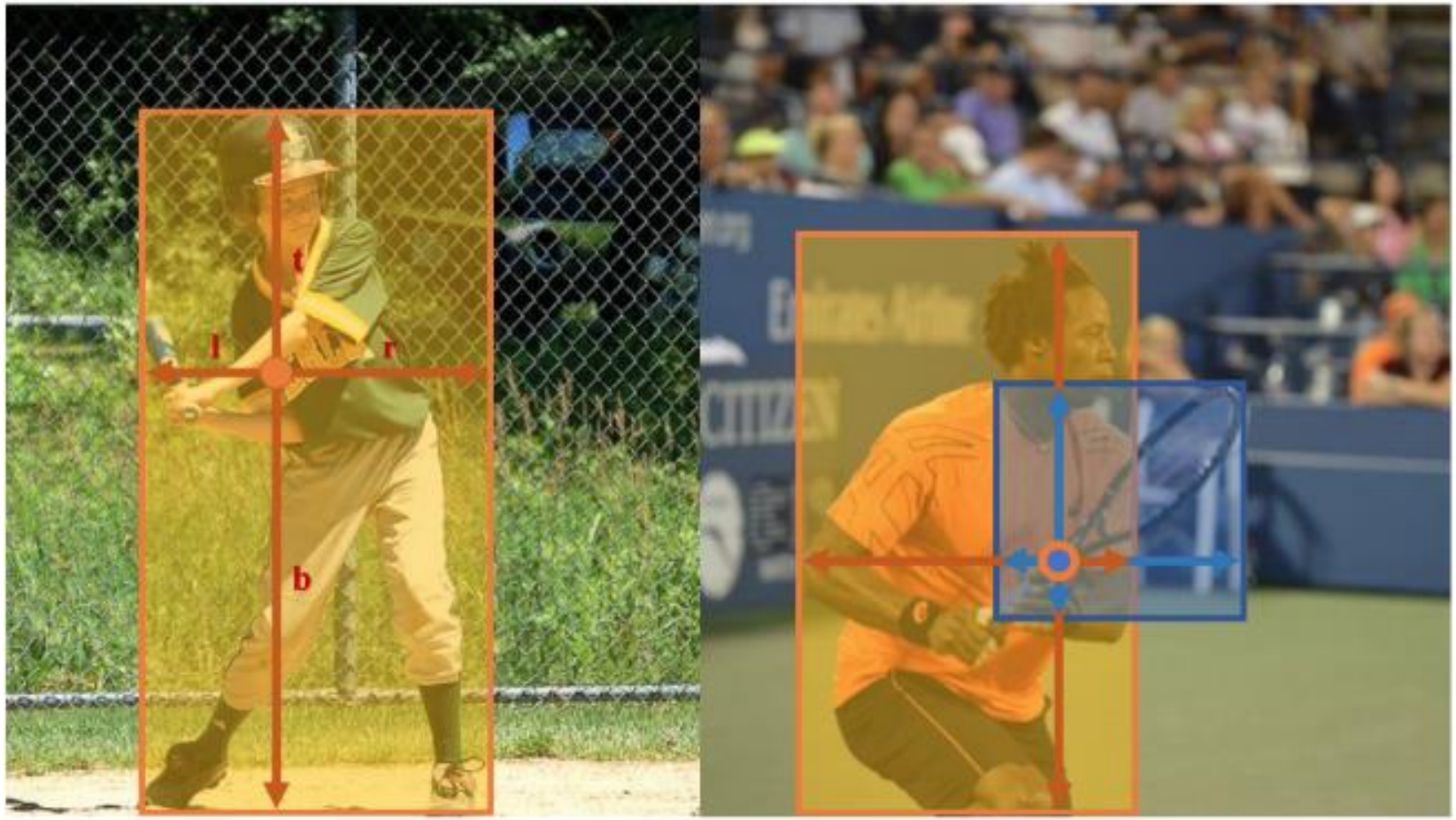
“FCOS”, One Shot Anchor-Free Object Detection



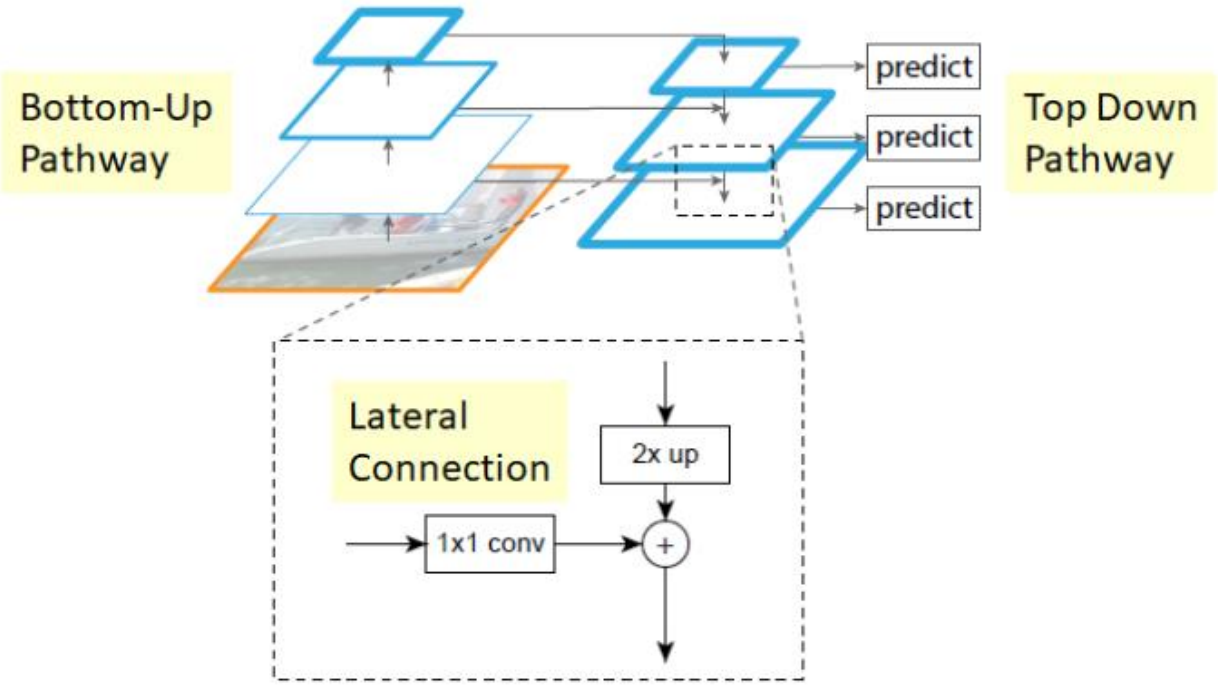
- FCOS is using **<u>Feature Pyramid Network(FPN)</u>** to create feature maps
- Adding head after every feature maps to train **Classification**, **Bounding Box Regression** and
- A noble index called **Center-ness**.

Center-ness : Label Mapping

- Left: FCOS is using (l, t, r, b) annotation → **positive sample**
- Right: ambiguous sample annotation →**ambiguous sample**



Feature Pyramid Network(FPN)



Structure of FPN (Photo by Sik-Ho Tsang)

- Feature Pyramid Network is a methods for creating levels of feature maps.
- **Adding (fusing) higher and lower level of features** (lower level features maps will become semantically strong)

Feature Pyramid Network(FPN) and FCOS

- The detection size of each layer will be different, **higher levels in charge of bigger object** and **lower level in charge for smaller object**, in order to divide the task for different layers.
- **FCOS treat every point in ground truth box as positive sample**. That results in lot of low-quality predicted bounding boxes produced by **locations far away from the center of an object.**

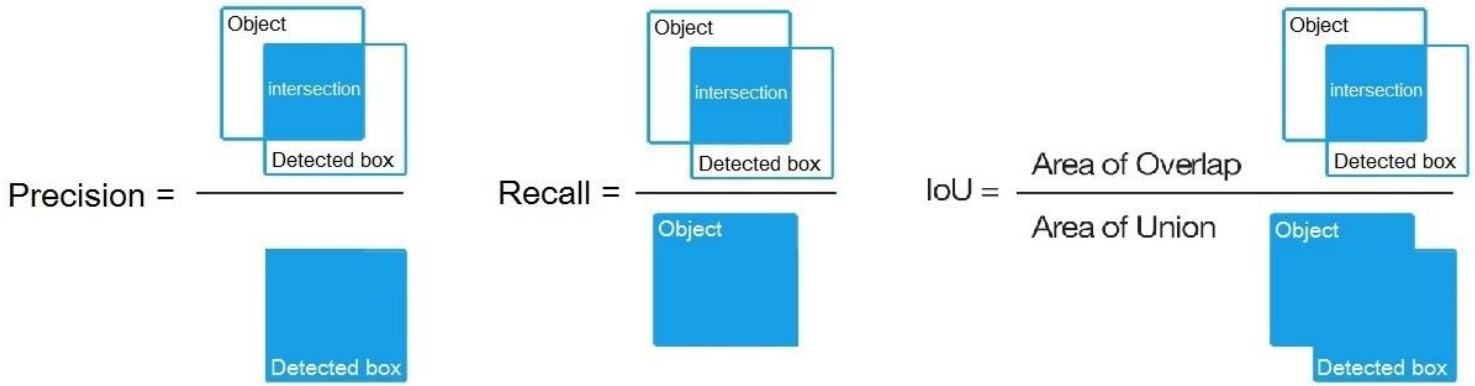
$$\text{centerness}^* = \sqrt{\frac{\min(l^*, r^*)}{\max(l^*, r^*)} \times \frac{\min(t^*, b^*)}{\max(t^*, b^*)}}.$$

FCOS

- Center-ness is between 0~1.
- For training using **BCELoss** (Binary Cross Entropy).
- For predicting, Center-ness can **multiplies to classification score to suppress the low-quality bounding boxes**, and
- Those boxes can easily remove by **Non Maximum Suppression**.

Results

- Performance metrics



Method	Backbone	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Two-stage methods:							
Faster R-CNN+++ [7]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w/ FPN [11]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [8]	Inception-ResNet-v2 [22]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w/ TDM [21]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	52.1
One-stage methods:							
YOLOv2 [18]	DarkNet-19 [18]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [15]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [4]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [12]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet [12]	ResNeXt-32x8d-101-FPN	40.8	61.1	44.1	24.1	44.2	51.2
CornerNet [10]	Hourglass-104	40.5	56.5	43.1	19.4	42.7	**53.9**
FCOS (ours)	ResNet-101-FPN	41.0	60.7	44.1	24.0	44.1	51.0
FCOS (ours)	ResNeXt-32x8d-101-FPN	**42.1**	**62.1**	**45.2**	**25.6**	**44.9**	52.0

Summary

- We have learned detection models
- Detection with Anchors:
  - Region-based Convolutional Neural Network (RCNN)
    - Fast R-CNN
    - Faster R-CNN
  - R-FCN, or **R**egion-based **F**ully **C**onvolutional **N**et
  - SSD
  - YOLO
- Anchor free detectors:
  - FCOS: Fully Convolutional One-Stage Object Detection

- What's next?
  - RNN